

TASK CODE :- FW10779

Main.c

```
#include "gpio_drive.h" // this is the library used to initiliasse the GPIO pins
```

```
#include "stm32f10x.h" // standard library used to access the registers
```

```
#include "systick.h" // for delay or handling perfect timings
```

```
unsigned long last_press_time_0 = 0;
```

```
unsigned long last_press_time_1 = 0;
```

```
unsigned long last_press_time_2 = 0;
```

```
char count = 0;
```

```
int main() {
```

```
    systick_init(); //for starting the systick timer
```

```
    //setting up the pin
```

```
    //initialising all the GPIO for buttons and LED
```

```
    init_GP(PA,10,OUT50,O_GP_PP);
```

```
    init_GP(PA,11,OUT50,O_GP_PP);
```

```
    init_GP(PA,12,OUT50,O_GP_PP);
```

```
    init_GP(PA,0,IN,I_PP); // pin used for button
```

```
    systick_inr_start(); //starting with the systick inorder to use  
    milliseconds to avoid debounce
```

```
//setting up the interrupt
```

```
RCC->APB2ENR |= 1; //enabling the alternative function
```

```
//registering the interrupt in NVIC and setting other important  
regsiters such IMR
```

```
__disable_irq();
```

```
AFIO->EXTICR[0] = 0x0000;
```

```
EXTI->IMR |= 0x1;
```

```
EXTI->RTSR |= 0x1;
```

```
NVIC_EnableIRQ(EXTI0_IRQn);
```

```
__enable_irq();
```

```
// I have put this code in while as I need to run it multiple times  
it is out of while then this run only for once and then will stop and since  
initial condition for count = 0 no effect
```

```
while(1)
```

```
{
```

```
    if(count == 1)
```

```
{
```

```
    W_GP(PA,10,1);
```

```
    W_GP(PA,11,0);
```

```
    W_GP(PA,12,0);
```

```
}
```

```
else if(count == 2)
```

```
{
```

```

        W_GP(PA,10,0);

        W_GP(PA,11,1);

        W_GP(PA,12,0);
    }
    else if(count == 3)
    {
        W_GP(PA,10,0);

        W_GP(PA,11,0);

        W_GP(PA,12,1);
    }
    else if(count == 4)
    {
        W_GP(PA,10,1);

        W_GP(PA,11,1);

        W_GP(PA,12,1);
    }
    else if(count == 5)
    {
        count = 0;

        //here all the leds are turned off so that completion of
pattern can be properly judged

        W_GP(PA,10,0);

        W_GP(PA,11,0);

        W_GP(PA,12,0);
    }

```

```
}
```

```
}
```

// making the function that will execute when interrupt is being received from GPIO

```
void EXTI0_IRQHandler()
```

```
{
```

```
    unsigned long current_ms = get_current_ms();
```

```
    if ((current_ms - last_press_time_0) > 50){
```

```
        count++;
```

```
    }
```

```
    EXTI->PR |= (1 << 0); // I need to clear this flag else I will be stuck in here
```

```
}
```

Gpio_drive.c

```
#include"gpio_drive.h"
```

```
int i1;
```

// this is the initialising function here offset and used for the ease and reduce the execution time

```
void init_GP(unsigned short port,unsigned short pin,unsigned short
dir,unsigned short opt)
{
    unsigned short port_offset = 0x04;
    unsigned short offset = 0x00;
    unsigned short tpin = pin;
    volatile unsigned long * CR = 0;
    if (pin >7)
    {
        tpin -= 8; //this will map (0-7) with (8-15) so to use same logic
        offset = 0x01;
    }

    if(port == 1)
    {
        RCC_APB2ENR |= port_offset;
        CR = (volatile unsigned long *) (&GPIO_A + offset);
    }
    else if(port == 2)
    {
        RCC_APB2ENR |= (port_offset << 1);
        CR = (volatile unsigned long *) (&GPIO_B + offset);
    }
}
```

```

    }
    else if(port == 3)
    {
        RCC_APB2ENR |= (port_offset << 2);
        CR = (volatile unsigned long *) (&GPIO_C + offset);
    }

    *CR &= ~(0xf<<(tpin)*4); //reseting the target pin

    *CR |= ((dir<<(tpin)*4) | (opt<<(tpin*4+2))); // this for adding the
option and the direction bits

}

//this will read the state of the GPIO if the GPIO is set in input mode used to
check the connection of the button

int R_GP(unsigned short port, unsigned short pin)
{
    volatile unsigned long *IDR = 0; // initialising it to 0 so that it should
not point in any random direction at any point of time in code

    unsigned short offset = 0x02; // these value is decided from the
register map and from thier by shifting 2 from the main address we get to
IDR

```

```
int state; //finally needed variable
```

```
if(port == 1)
```

```
{
```

```
    IDR = (volatile unsigned long *) (&GPIO_A + offset);
```

```
}
```

```
else if (port == 2)
```

```
{
```

```
    IDR = (volatile unsigned long *) (&GPIO_B + offset);
```

```
}
```

```
else if (port == 3)
```

```
{
```

```
    IDR = (volatile unsigned long *) (&GPIO_C + offset);
```

```
}
```

```
    state = ((*IDR & (1 << pin))>>pin); // we are moving back with pin so  
that at the end bit it is 0 or 1
```

```
    return state;
```

```
}
```

```
//this will write any value to GPIO
```

```
void W_GP(unsigned short port , unsigned short pin, unsigned short state)
```

```
{
```

```

unsigned short offset = 0x03;
volatile unsigned long *ODR = 0;

if(port == 1)
{
    ODR = (volatile unsigned long *) (&GPIO_A + offset);
}
else if (port == 2)
{
    ODR = (volatile unsigned long *) (&GPIO_B + offset);
}
else if (port == 3)
{
    ODR = (volatile unsigned long *) (&GPIO_C + offset);
}

state ? (*ODR |= (state<<pin)) :    (*ODR &= ~(1<<pin));
}

```

Gpio_drive.h

```
#include "stm32f10x.h"
```

```
//main bus register
```

```
#define RCC_APB2ENR (*(volatile unsigned long *) 0x40021018) //using
the RCC register along with the offset of APB2 to enable the pin
```

```
//GPIO registers
```

```
#define GPIO_A    (*(volatile unsigned long *) 0x40010800)
```



```
#define GPIO_B    (*((volatile unsigned long *) 0x40010C00))
```

```
#define GPIO_C    (*((volatile unsigned long *) 0x40011000))
```

```
//List of port
```

```
#define PA 1
```

```
#define PB 2
```

```
#define PC 3
```

```
//setting the directions
```

```
#define IN  0 //using for input
```

```
#define OUT10 1 //using output port at 10MHZ
```

```
#define OUT2 2 //using output port at 2MHZ
```

```
#define OUT50 3 //using output port at 50MHZ
```

```
//direction options for input
```

```
#define I_AN 0
```

```
#define I_F 1
```

```
#define I_PP 2
```

```
//direction option for output
```

```
#define O_GP_PP 0
```

```
#define O_GP_OD 1
```

```
#define O_AF_PP 2
```

```
#define O_AF_OD 3
```

```
void init_GP(unsigned short port,unsigned short pin,unsigned short
dir,unsigned short opt);

int R_GP(unsigned short port, unsigned short pin);

void W_GP(unsigned short port , unsigned short pin, unsigned short val);

void toggle_GP(unsigned short port, unsigned short pin);

void Digital_Input(unsigned short PORT,unsigned short PIN);

void Digital_Output(unsigned short PORT,unsigned short PIN);
```

systick.c

```
#include "systick.h"

#include "stm32f10x.h"

volatile unsigned long msTicks = 0;

//this is the intialsing function

void systick_init(void)
{
    SysTick->CTRL = 0;

    SysTick->LOAD = 0x00FFFFFF;

    SysTick->VAL = 0;
```

```

        SysTick->CTRL = 5;
    }

// for using the interrupt that will be generated on completion of the count
void systick_inr_start()
{
    __disable_irq();
    SysTick->CTRL = 0;
    SysTick->LOAD = 72000 - 1; //-1 because I am considering 0 as well
    and this is for 1ms time
    SysTick->VAL = 0;
    SysTick->CTRL |= 7;
    __enable_irq();
}

//this function will be executed when interrupt will be generated
void SysTick_Handler(void) {
    msTicks++;
}

//function to get the current value
unsigned long get_current_ms(void) {
    return msTicks;
}

```

Systick.h

```
void DelayMillis(void);
```

```
void DelayMs(unsigned long t);
```

```
void systick_init(void);
```

```
void systick_inr_start();
```

```
unsigned long get_current_ms(void);
```